

Contents

Introduction.....	1
Basic Usage.....	1
Customization	2
Y axis	2
X axis	2
Bar.....	3
Performing Calculations Before Plotting.....	4
Data types	4

Introduction

This document explains how to use the Simple Bar Graph asset. It is compatible with **Unity 2020.x or later**.

The prefab samples are present in folders of examples:

(../Simple Bar Graph/Examples/Example n).

Basic Usage

The line graph is represented by the *BarGraph* class. To draw the graph, a dataset (*IBarGraphData*) is required as the first parameter of the *PlotGraph* method. **Before drawing**, the graph is automatically cleared to ensure no overlap with previous data.

Example:

```
[SerializeField] private BarGraph barGraph;
[SerializeField] private SerialiseBarGraphData data;

public void Plot()
{
    barGraph.PlotGraph(data);
}
```

Customization

Y axis

According to **Discretization and MinMaxOnGraph** defined in the **Bar Graph**, specific value on axis is set, while other UI elements are set up by **anchors**. For exact rendering of the value, base class **YValueRender** should be used.

Example:



```
public class OnlyTextYValueRender : YValueRender
{
    [SerializeField] private string format = "{0:F2}";
    private TMP_Text _text;

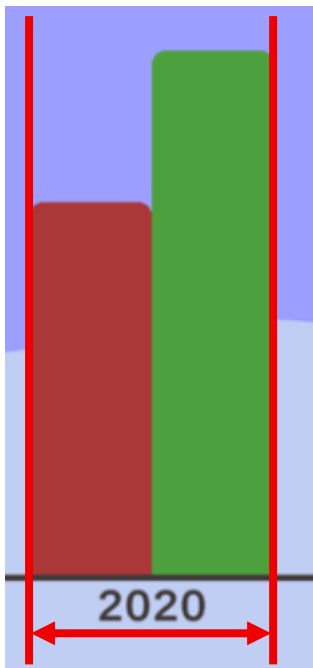
    private void Awake()
    {
        _text = GetComponentInChildren<TMP_Text>();
    }

    public override void Render(float data)
    {
        if (_text != null)
        {
            _text.text = string.Format(format, data);
        }
    }
}
```

X axis

The width of the **Bar Set** is defined by the width of the prefab that renders the bottom value. For exact rendering of the value, base class **XValueRender** should be used.

Example:



```
public class OnlyTextXValueRender : XValueRender
{
    private TMP_Text _text;

    private void Awake()
    {
        _text = GetComponentInChildren<TMP_Text>();
    }

    public override void Render(string data, ICollection<IBar> bars)
    {
        if (_text != null)
        {
            _text.text = data;
        }
    }
}
```

Bar

For exact rendering of a **single bar**, the **BarRender** class should be used.

Example:

```
public class OnlyImageBarRender : BarRender
{
    [SerializeField] private List<Color> colors;
    private Image[] _barCopy;

    private void Awake()
    {
        _barCopy = GetComponentsInChildren<Image>();
    }

    public override void Render(IBar bar, Vector2 minMax)
    {
        var color = colors[bar.ID];
        foreach (var image in _barCopy)
        {
            image.color = color;
        }
    }
}
```

As shown in the example above, each bar's color is determined by its ID and the predefined color set.

If a more detailed prefab customization is required, a **SetBarPrefab** method should be used to apply the configured prefab.

Performing Calculations Before Plotting

If you need to run calculations after determining the dataset's minimum and maximum values, you can use the optional **criticalValuesFound** delegate parameter in the **PlotGraph** method. This delegate allows you to specify which minimum and maximum values will be shown on the graph via the **MinMaxOnGraph** property.

Data types

Each data set should realize the following interfaces:

```
public interface IBarGraphData
{
    public IDictionary<string, ICollection<IBar>> BarSets { get; }
    public Vector2 GetMinMaxValues();
}

public interface IBar
{
    public int ID { get; }
    public float Value { get; }
}
```

Each **IBar** is responsible for one single bar, while *BarSets* property contains multiple bars

under the single X Value, shown as the bottom of the graph.